

AGENTS AND PARALLEL WORK

Orchestrate subagents at scale with dynamic workflows

Dynamic workflows orchestrate many subagents from a script Claude writes and you can rerun. Use them for codebase audits, large migrations, and cross-checked research.

 Dynamic workflows require Claude Code v2.1.154 or later and are available on all paid plans, with Anthropic API access, and on Amazon Bedrock, Google Cloud Vertex AI, and Microsoft Foundry. On Pro, turn them on from the Dynamic workflows row in `/config`.

A dynamic workflow is a JavaScript script that orchestrates subagents at scale. Claude writes the script for the task you describe, and a runtime executes it in the background while your session stays responsive.

Reach for a workflow when a task needs more agents than one conversation can coordinate, or when you want the orchestration codified as a script you can read and rerun. Examples include a codebase-wide bug sweep, a 500-file migration, a research question that needs sources cross-checked against each other, and a hard plan worth drafting from several independent angles before you commit to one.

When to use a workflow

Subagents, skills, agent teams, and workflows can all run a multi-step task. The difference is who holds the plan:

	Subagents	Skills	Agent teams	Workflows
What it is	A worker Claude spawns	Instructions Claude follows	A lead agent supervising peer sessions	A script the runtime executes
Who decides what runs next	Claude, turn by turn	Claude, following the prompt	The lead agent, turn by turn	The script
Where intermediate	Claude's context	Claude's context	A shared task list	Script variables

	Subagents	Skills	Agent teams	Workflows
results live	window	window		
What's repeatable	The worker definition	The instructions	The team definition	The orchestration itself
Scale	A few delegated tasks per turn	Same as subagents	A handful of long-running peers	Dozens to hundreds of agents per run
Interruption	Restarts the turn	Restarts the turn	Teammates keep running	Resumable in the same session

A workflow moves the plan into code. With subagents, skills, and agent teams, Claude is the orchestrator: it decides turn by turn what to spawn or assign next, and every result lands in a context window. A workflow script holds the loop, the branching, and the intermediate results itself, so Claude's context holds only the final answer.

Moving the plan into code also lets a workflow apply a repeatable quality pattern, not just run more agents: it can have independent agents adversarially review each other's findings before they're reported, or draft a plan from several angles and weigh them against each other, so you get a more trustworthy result than a single pass.

Run a bundled workflow

The quickest way to see a workflow in action is to run `/deep-research`, the **built-in workflow** Claude Code includes for investigating a question across many sources. You'll see agents work through a set of phases in the background while your session stays free, and get one report at the end instead of a turn-by-turn transcript.

1 Run the workflow

Run `/deep-research` with a question you want investigated. It fans out web searches across several angles, fetches and cross-checks the sources it finds, and synthesizes a cited report.

```
/deep-research What changed in the Node.js permission model between v20 and v22?
```

2 Allow workflows

Claude Code asks whether to allow the workflow. Select **Yes** to continue. The exact prompt depends on your permission mode. See [Approve the plan before it runs](#) for the per-mode options.

3 **Watch progress**

The run starts in the background. Run `/workflows` , use the arrow keys to select the run, and press Enter to open its progress view:

```
/workflows
```

The view shows each phase with its agent count, token total, and elapsed time. Drill into any phase to see its agents and what each one found. See [Watch the run](#) for the full set of controls.

You can also watch from the task panel below the input box: a one-line progress summary appears there while the run is going. Press the down arrow to focus it, then Enter to expand.

4 **Read the report**

When the run finishes, the report lands in your session. It cites the sources each claim came from, with claims that didn't survive cross-checking already filtered out.

To run a workflow for your own task, [have Claude write one](#), and once a run does what you wanted you can [save it](#) as a command of your own.

Bundled workflows

Claude Code includes `/deep-research` as a built-in workflow:

Command	What it does
<code>/deep-research</code> <code><question></code>	Fans out web searches on a question across several angles, fetches and cross-checks the sources it finds, votes on each claim, and returns a cited report with claims that didn't survive cross-checking filtered out. Requires the WebSearch tool to be available

[Workflows you save](#) yourself become commands the same way and appear in `/` autocomplete alongside the bundled ones.

Watch the run

Workflows run in the background, so the session stays responsive while agents work. Run `/workflows` at any time to list running and completed workflows, then select one to open its progress view.

```
/workflows
```

The progress view shows each phase with its agent counts, token totals, and elapsed time. The footer lists the key for each action:

Key	Action
<code>↑ / ↓</code>	Select a phase or agent
<code>Enter</code> or <code>→</code>	Drill into the selected phase, then into an agent to read its prompt, recent tool calls, and result
<code>Esc</code>	Back out one level
<code>j / k</code>	Scroll within the agent detail when it overflows
<code>f</code>	Filter the agent list in the selected phase by status. Press again to cycle
<code>p</code>	Pause or resume the run
<code>x</code>	Stop the selected agent, or stop the whole workflow when focus is on the run
<code>r</code>	Restart the selected running agent
<code>s</code>	<u>Save</u> the run's script as a command

Have Claude write a workflow

You can have Claude write a workflow for your task in two ways:

- Ask for a workflow in your prompt, either in your own words or by including the keyword `ultracode` , and Claude writes one for the task.
- Let Claude decide with ultracode: set `/effort ultracode` and Claude plans a workflow for every substantive task in the session.

You can also run a workflow command that already exists: a bundled workflow like `/deep-research` , or one you've saved.

Ask for a workflow in your prompt

To run a single task as a workflow without changing the session's effort level, include the keyword `ultracode` in your prompt. Asking in your own words, for example “use a workflow” or “run a workflow”, also works: Claude treats a direct request as the same opt-in. Before v2.1.160 the literal trigger keyword was `workflow` ; natural-language requests work in both versions.

```
ultracode: audit every API endpoint under src/routes/ for missing
auth checks
```

Claude Code highlights the keyword in your input and Claude writes a workflow script for the task instead of working through it turn by turn. If you didn't mean to start a workflow, press `Option+W` on macOS or `Alt+W` on Windows and Linux to dismiss the highlight for this prompt, or press backspace while the cursor is right after the highlighted keyword. To stop the keyword from triggering at all, turn off Ultracode keyword trigger in `/config` .

If the run does what you wanted, you can **save it as a command** afterward.

If you already have an orchestrator built another way, such as a folder of subagent prompts or a skill that fans work out, you can point Claude at it and ask for a workflow that does the same thing.

Let Claude decide with ultracode

Ultracode is a Claude Code setting that combines `xhigh` **reasoning effort** with automatic workflow orchestration. With it on, Claude plans a workflow for each substantive task instead of waiting for you to ask.

```
/effort ultracode
```

With ultracode on, Claude decides when a task warrants a workflow. A single request can turn into several workflows in a row: one to understand the code, one to make the change, and one to verify it. This applies to every task in the session, so each request uses more tokens and takes longer than at lower effort levels.

Ultracode lasts for the current session and resets when you start a new one. Drop back with `/effort high` when you return to routine work. It's available on models that support `xhigh` **effort**; on other models the `/effort` menu doesn't offer it.

Approve the plan before it runs

In the CLI, the per-run prompt shows the planned phases and these options:

- **Yes, run it:** start the run
- **Yes, and don't ask again for** `<name>` **in** `<path>` : start, and skip this prompt for this workflow in this project from now on
- **View raw script:** read the script before deciding
- **No:** cancel

`Ctrl+G` opens the script in your editor. `Tab` lets you adjust the prompt before the run starts.

Whether you see this prompt depends on your permission mode:

Permission mode	When you're prompted
Default, accept edits	Every run, unless you've selected Yes, and don't ask again for that workflow in this project
Auto	First launch only. Any Yes records consent in your user settings, and later launches start without prompting. Skipped entirely when ultracode is on
Bypass permissions, <code>claude -p</code> , Agent SDK	Never. The run starts immediately

In the Desktop app, an approval card shows the workflow name, the phase list, and a token-usage caution, with **Once**, **Always**, and **Deny** actions. The progress view appears in the Background tasks side pane.

Your permission mode controls only the launch prompt above. The subagents the workflow spawns always run in `acceptEdits` mode and inherit your tool allowlist, regardless of your session's mode. File edits are auto-approved.

Shell commands, web fetches, and MCP tools that aren't in your allowlist can still prompt you mid-run. To avoid this on a long run, add the commands the agents need to your allowlist before starting.

In `claude -p` and the Agent SDK there is no one to prompt, so tool calls follow your configured permission rules without interactive confirmation.

Save the workflow for reuse

When Claude writes a workflow for a task you'll repeat, you can save that run's script as a command.

A process like a review you run on every branch then runs the same orchestration each time.

Run `/workflows`, select the run you want to keep, and press `s`. In the save dialog, Tab toggles between the two save locations:

- `.claude/workflows/` in your project: shared with everyone who clones the repo
- `~/.claude/workflows/` in your home directory: available in every project, visible only to you

Press Enter to save. The workflow runs as `/<name>` in future sessions from either location.

In a monorepo with several `.claude/` directories, you can keep workflows alongside the package they apply to. As of v2.1.178, saving to the project location writes to the closest

`.claude/workflows/` directory that already exists between your working directory and the repository root, or to the repository root if none exists yet. Project workflows also load from every `.claude/workflows/` along that path, and when more than one defines the same name Claude Code runs the one closest to the working directory.

If a project workflow and a personal workflow share a name, the project one runs.

Pass input to a saved workflow

A saved workflow can accept input through the `args` parameter. The script reads it as a global named `args`. Use this to supply a research question, a list of target paths, or a configuration object at invocation time instead of editing the script for each run.

The following prompt runs a saved workflow with a list of issue numbers:

```
> Run /triage-issues on issues 1024, 1025, and 1030
```

Claude passes the list as structured data, so the script can call array and object methods on `args` directly without parsing it first. If `args` is omitted, the global is `undefined` inside the script.

How a workflow runs

The workflow runtime executes the script in an isolated environment, separate from your conversation. Intermediate results stay in script variables instead of landing in Claude's context.

Every run writes its script to a file under your session's directory in `~/.claude/projects/`. Claude receives the path when the run starts, so you can ask for it. You can open that file to read the

orchestration Claude wrote, diff it against a previous run’s script, or edit it and ask Claude to relaunch from the edited version.

The runtime tracks each agent’s result as the run progresses, which is what makes a run resumable within the same session.

Behavior and limits

The runtime applies the following constraints:

Constraint	Why
No mid-run user input	Only agent permission prompts can pause a run. For sign-off between stages, run each stage as its own workflow
No direct filesystem or shell access from the workflow itself	Agents read, write, and run commands. The script coordinates the agents
Up to 16 concurrent agents, fewer on machines with limited CPU cores	Bounds local resource use
1,000 agents total per run	Prevents runaway loops

Manage runs

Once a run starts, you manage it from the `/workflows` view, or by expanding its progress line in the task panel below the input box.

Resume after a pause

If you stop a run, you can resume it: agents that already completed return their cached results, and the rest run live. Resume a paused run from `/workflows` by selecting it and pressing `p` , or ask Claude to relaunch the workflow with the same script.

Resume works within the same Claude Code session. If you exit Claude Code while a workflow is running, the next session starts the workflow fresh.

Cost

A workflow spawns many agents, so a single run can use meaningfully more tokens than working through the same task in conversation. Runs count toward your plan’s usage and rate limits like any

other session.

To gauge the spend before committing to a large task, run the workflow on a small slice first: one directory instead of the whole repo, or a narrow question instead of a broad one. The `/workflows` view shows each agent's token usage as the run progresses, and you can stop the run there at any time without losing completed work. The runtime's **agent caps** limit how many agents a single run can spawn, which bounds the cost of a runaway script.

Every agent in a workflow uses your session's model unless the script routes a stage to a different one. To control the model cost:

- Check `/model` before a large run if you usually switch to a smaller model for routine work
- Ask Claude to use a smaller model for stages that don't need the strongest one when you describe the task

Turn workflows off

Workflows are available in the CLI, the Desktop app, the IDE extensions, **non-interactive mode** with `claude -p`, and the **Agent SDK**. The same disable settings apply on every surface.

To turn workflows off for yourself:

- Toggle Dynamic workflows off in `/config`. Persists across sessions.
- Set `"disableWorkflows": true` in `~/.claude/settings.json`. Persists across sessions.
- Set `CLAUDE_CODE_DISABLE_WORKFLOWS=1`. Read at startup, so it applies wherever you set it.

To turn workflows off for your whole organization, set `"disableWorkflows": true` in **managed settings**, or use the toggle on the **Claude Code admin settings** page.

When workflows are disabled, the bundled workflow commands are unavailable, the `ultracode` keyword no longer triggers a run, and `ultracode` is removed from the `/effort` menu.

Related resources

- **Run agents in parallel**: compare subagents, agent view, agent teams, and workflows
- **Create custom subagents**: the worker primitive workflows orchestrate
- **Manage costs**: how multi-agent runs count toward usage limits

